# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- now we know the major components of attention mechanisms under the framework in Figure 3
- to recapitulate, the interactions between queries (volitional cues) and keys (nonvolitional cues) result in *attention pooling*
- the attention pooling selectively aggregates values (sensory inputs) to produce the output
- in this section, we will describe attention pooling in greater detail, to give a high-level view of how attention mechanisms work in practice
- specifically, the Nadaraya-Watson kernel regression model, proposed in 1964, is a simple yet complete example for demonstrating machine learning with attention mechanisms

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- to keep things simple, let us consider the following regression problem: given a dataset of input-output pairs $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, how to learn $f$ to predict the output $\hat{y} = f(x)$ for any new input $x$?
- we begin with perhaps the world's "dumbest" estimator for this regression problem: using average pooling to average over all the training outputs:

$$f(x) = \frac{1}{n} \sum_{i=1}^{n} y_i. \tag{1}$$

- obviously, average pooling omits the inputs $x_i$
- a better idea was proposed by Nadaraya and Watson, namely to weigh the outputs $y_i$ according to their input locations:

$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_{j=1}^{n} K(x - x_j)} y_i, \tag{2}$$

  where $K$ is a *kernel*
- the estimator in (2) is called *Nadaraya-Watson kernel regression*; here, we will not dive into details of kernels

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- recall the framework of attention mechanisms in Figure 3; from the perspective of attention, we can rewrite (2) in a more generalized form of *attention pooling*:

$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i, \tag{3}$$

  where $x$ is the *query* and $(x_i, y_i)$ is the *key-value* pair
- comparing (3) and (1), the attention pooling here is a weighted average of values $y_i$
- the *attention weight* $\alpha(x, x_i)$ in (3) is assigned to the corresponding value $y_i$ based on the interaction between the query $x$ and the key $x_i$, modeled by $\alpha$
- for any query, its attention weights over all the key-value pairs are a valid probability distribution: they are non-negative and sum up to one

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- to gain intuitions of attention pooling, we can consider a Gaussian kernel defined as:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

- plugging the Gaussian kernel into (3) and (2) gives:

$$\begin{aligned} f(x) &= \sum_{i=1}^{n} \alpha(x, x_i) y_i \\ &= \sum_{i=1}^{n} \frac{\exp\left(-\frac{1}{2}(x - x_i)^2\right)}{\sum_{j=1}^{n} \exp\left(-\frac{1}{2}(x - x_j)^2\right)} y_i \\ &= \sum_{i=1}^{n} \text{softmax}\left(-\frac{1}{2}(x - x_i)^2\right) y_i. \end{aligned} \tag{4}$$

- in (4), a key $x_i$ that is closer to the given query $x$ will get *more attention* via a *larger attention weight* assigned to the key's corresponding value $y_i$

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- notably, Nadaraya-Watson kernel regression is a nonparametric model; thus (4) is an example of *nonparametric attention pooling*
- nonparametric Nadaraya-Watson kernel regression enjoys the *consistency* benefit: given enough data, this model converges to the optimal solution
- nonetheless, we can easily integrate learnable parameters into attention pooling
- as an example, slightly different from (4), in the following, the distance between the query $x$ and the key $x_i$ is multiplied by a learnable parameter $w$:

$$
\begin{aligned}
f(x) &= \sum_{i=1}^{n} \alpha(x, x_i) y_i \\
&= \sum_{i=1}^{n} \frac{\exp\left(-\frac{1}{2}((x - x_i)w)^2\right)}{\sum_{j=1}^{n} \exp\left(-\frac{1}{2}((x - x_j)w)^2\right)} y_i \\
&= \sum_{i=1}^{n} \operatorname{softmax}\left(-\frac{1}{2}((x - x_i)w)^2\right) y_i.
\end{aligned}
$$